API Groups

Before diving into Kubernetes authorization, it is essential to understand how API groups are organized. The Kubernetes API forms the backbone of all cluster interactions. Whether you are using the kubectl utility or directly accessing the API via REST, every operation communicates with the Kubernetes API server.

For example, to check the cluster version, you can send a request to the master node on the default port 6443 using:

```
curl https://kube-master:6443/version
```

The response will include version details similar to the example below:

```
{
  "major": "1",
  "minor": "13",
  "gitVersion": "v1.13.0",
  "gitCommit": "ddf47ac13c1a9483ea035a79cd7c10005ff21a6d",
  "gitTreeState": "clean",
  "buildDate": "2018-12-03T20:56:12Z",
  "goVersion": "go1.11.2",
  "compiler": "gc",
  "platform": "linux/amd64"
}
```

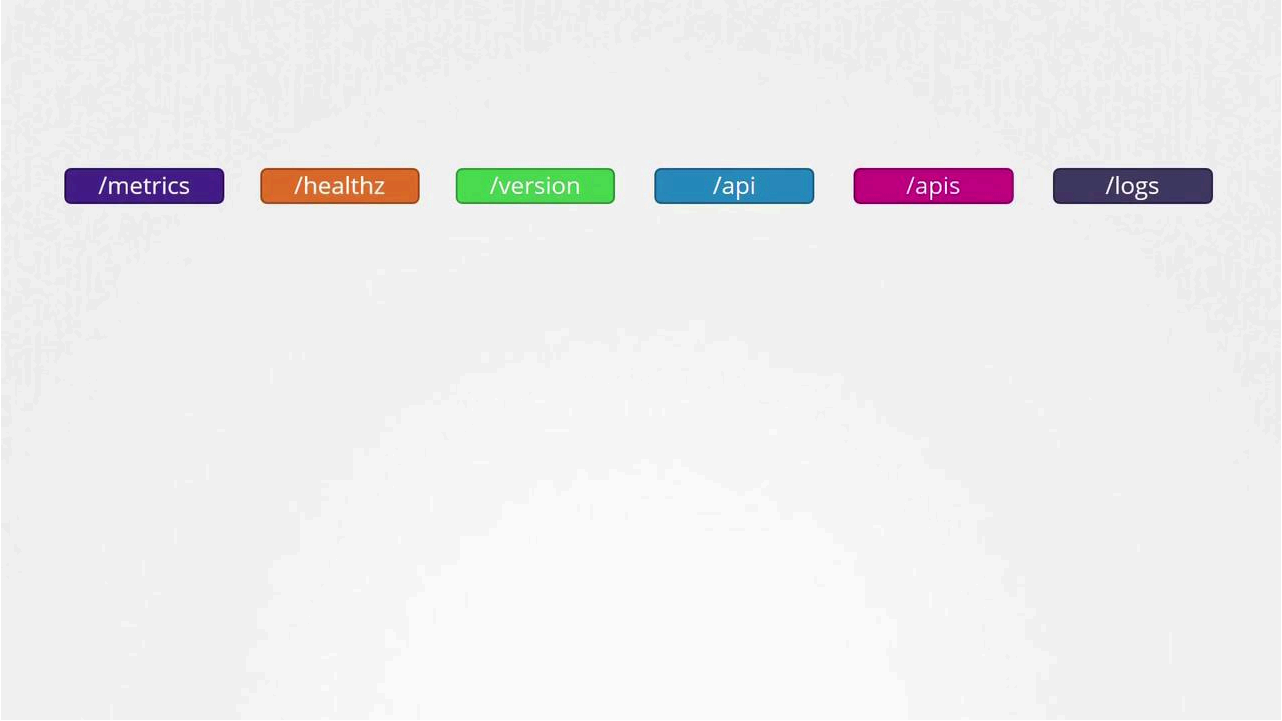
Similarly, to retrieve a list of pods, you can access:

```
curl https://kube-master:6443/api/v1/pods
```

This command returns a JSON response containing pod details. For example:


```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
    "resourceVersion": "153068"
  },
  "items": [
    {
      "metadata": {
        "name": "nginx-5c7588df-ghsbd",
        "generateName": "nginx-5c7588df-",
        "namespace": "default",
        "creationTimestamp": "2019-03-20T10:57:48Z",
        "labels": {
          "app": "nginx",
          "pod-template-hash": "5c7588df"
        },
        "ownerReferences": [
          {
            "apiVersion": "apps/v1",
            "kind": "ReplicaSet",
            "name": "nginx-5c7588df",
            "uid": "398ec179-4af9-11e9-beb6-020d3114c7a7",
            "controller": true,
            "blockOwnerDeletion": true
          }
        ]
      }
    }
  ]
}
```

Kubernetes organizes its API into multiple groups based on functionality. There are separate groups for metrics, health, version, logging, and more. The diagram below illustrates several of these endpoints:



The version API provides essential cluster version details, while the metrics and health APIs are used for monitoring cluster health. Additionally, the logs API integrates with third-party logging systems.

In this lesson, our focus is on the APIs that drive core cluster functionality, divided into two major categories:

1. **Core API Group**
   This group includes essential resources such as:

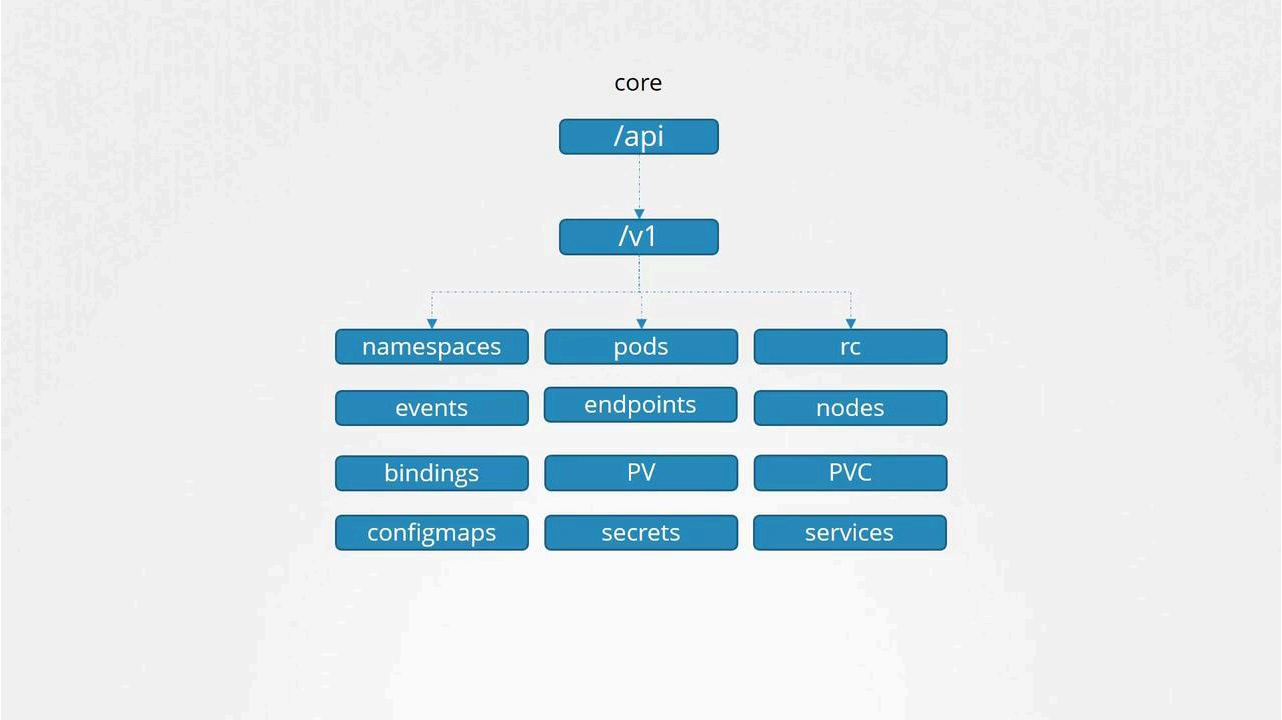
   - Namespaces
   - Pods
   - Replication Controllers
   - Events
   - Endpoints
   - Nodes

- Bindings
- Persistent Volumes
- Persistent Volume Claims
- Config Maps
- Secrets
- Services

2. **Named API Groups**
   These groups organize newer features and resources, such as:

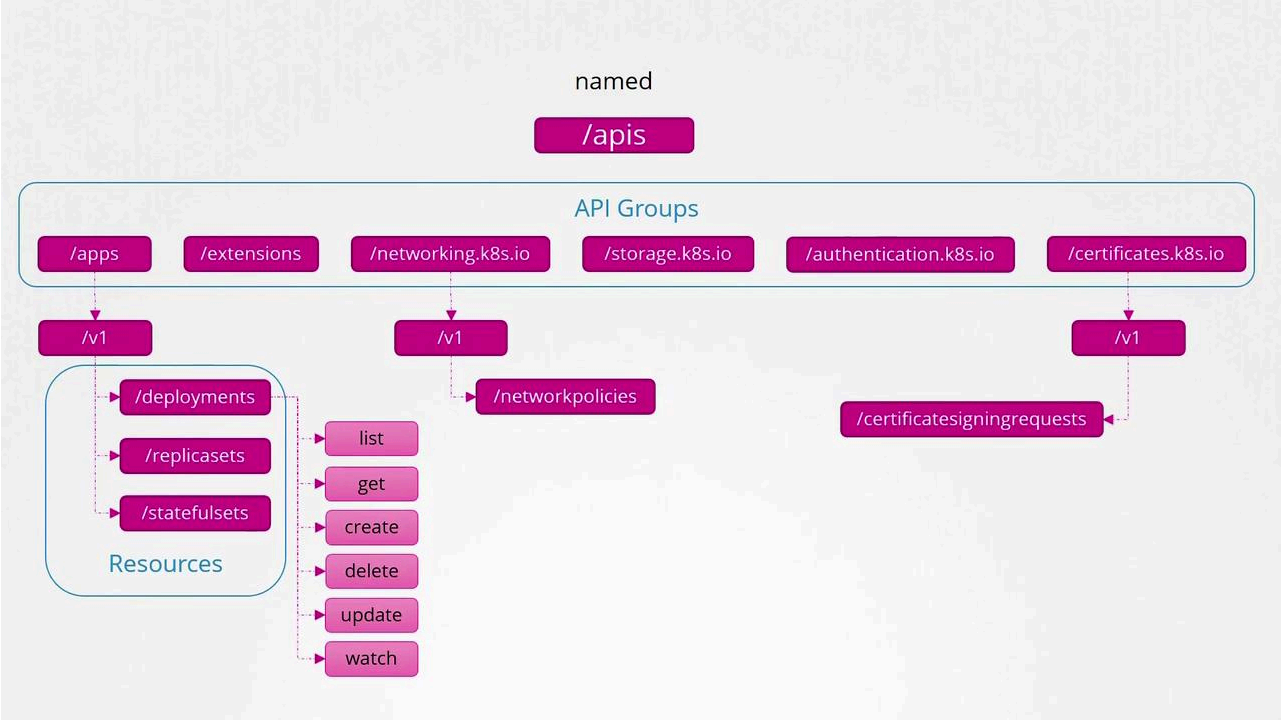
   - Apps (for Deployments, ReplicaSets, StatefulSets)
   - Extensions
   - Networking (for Network Policies)
   - Storage
   - Authentication
   - Authorization
   - And others (including certificate signing requests)

Below is a hierarchical view of core Kubernetes API resources, including namespaces, pods, nodes, and services:

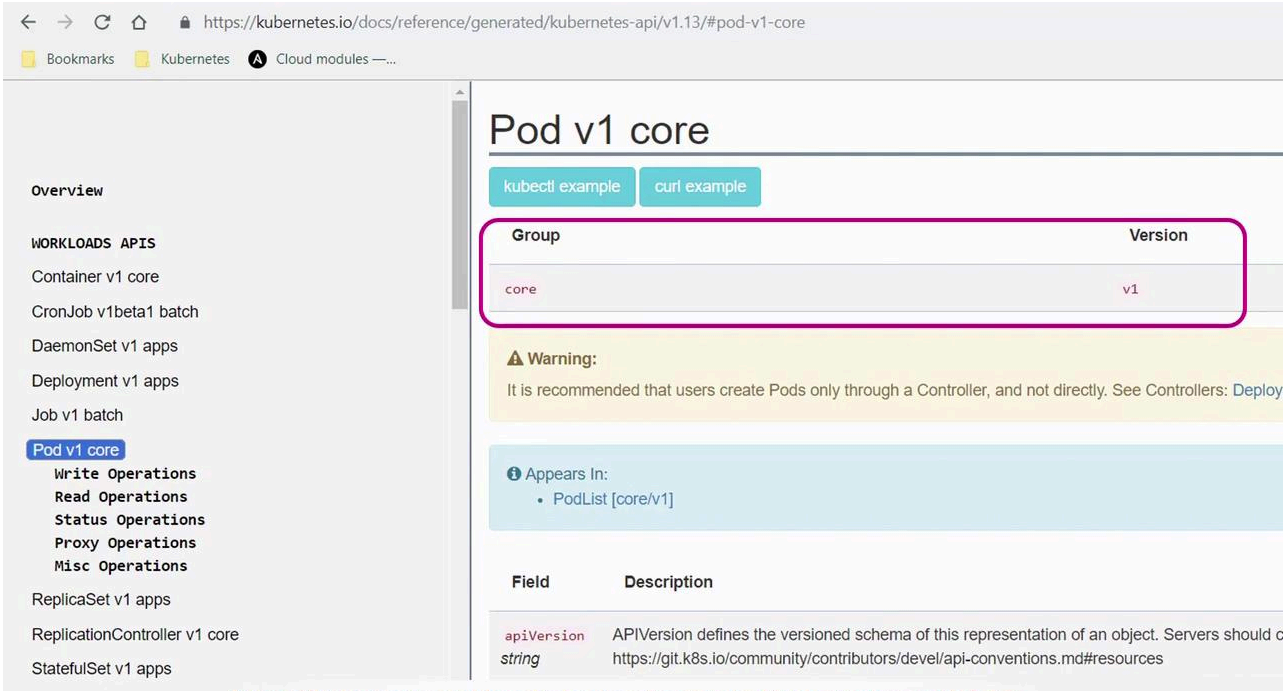


Within the named API groups, resources are further organized by category. For example, the "apps" group contains deployments, replica sets, and stateful sets. Similarly, the "networking" group includes network policies, and additional groups manage certificate signing requests and other resources.

Each Kubernetes resource supports a set of operations, known as verbs, which include actions such as listing, retrieving, creating, deleting, updating, and watching resources. The diagram below provides an overview of Kubernetes API groups, resources, and available actions:



For detailed information about each API group and resource, refer to the **Kubernetes API reference page**. Selecting an object on this page provides group-specific details. For example, the "Pod v1 core" documentation highlights that core APIs are part of the v1 group:



https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.13

You can also view API group information directly from your Kubernetes cluster. Simply accessing the API server at port 6443 without specifying a path will list all available API groups. For instance, running the command:

```
curl http://localhost:6443 -k
```

might return:

```
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis/",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json"
  ]
}
```

To filter the output for named API groups, you can use:

```
curl http://localhost:6443/apis -k | grep "name"
```

This command could output entries such as:

- "extensions"
- "apps"
- "events.k8s.io"
- "authentication.k8s.io"
- "authorization.k8s.io"
- "autoscaling"

- "batch"
- "certificates.k8s.io"
- "networking.k8s.io"
- "policy"
- "rbac.authorization.k8s.io"
- "storage.k8s.io"
- "admissionregistration.k8s.io"
- "apiregistration.k8s.io"
- "scheduling.k8s.io"

Access Restrictions

If you attempt to access the API directly using curl without proper authentication, you will receive a forbidden error message. For instance:

```
curl http://localhost:6443 -k
```

returns:

```
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "Forbidden: User \"system:anonymous\" cannot get path \"/\"",
  "reason": "Forbidden",
  "details": {},
  "code": 403
}
```

To authenticate, you can pass certificate files via the command line, or alternatively, use the kubectl proxy.

The **`kubectl proxy`** command launches a local HTTP proxy on port 8001 that uses the credentials and certificates from your kubeconfig file. This approach avoids the need to manually specify authentication parameters with curl. Once the proxy is running, you can access the API server by executing:

```
kubectl proxy
```

The output will indicate the proxy is running:

```
Starting to serve on 127.0.0.1:8001
```

Then, accessing the API proxy with:

```
curl http://localhost:8001 -k
```

will return:

```
{
  "paths": [
    "/api/",
    "/api/v1",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json"
  ]
}
```

Kube-proxy vs kubectl proxy

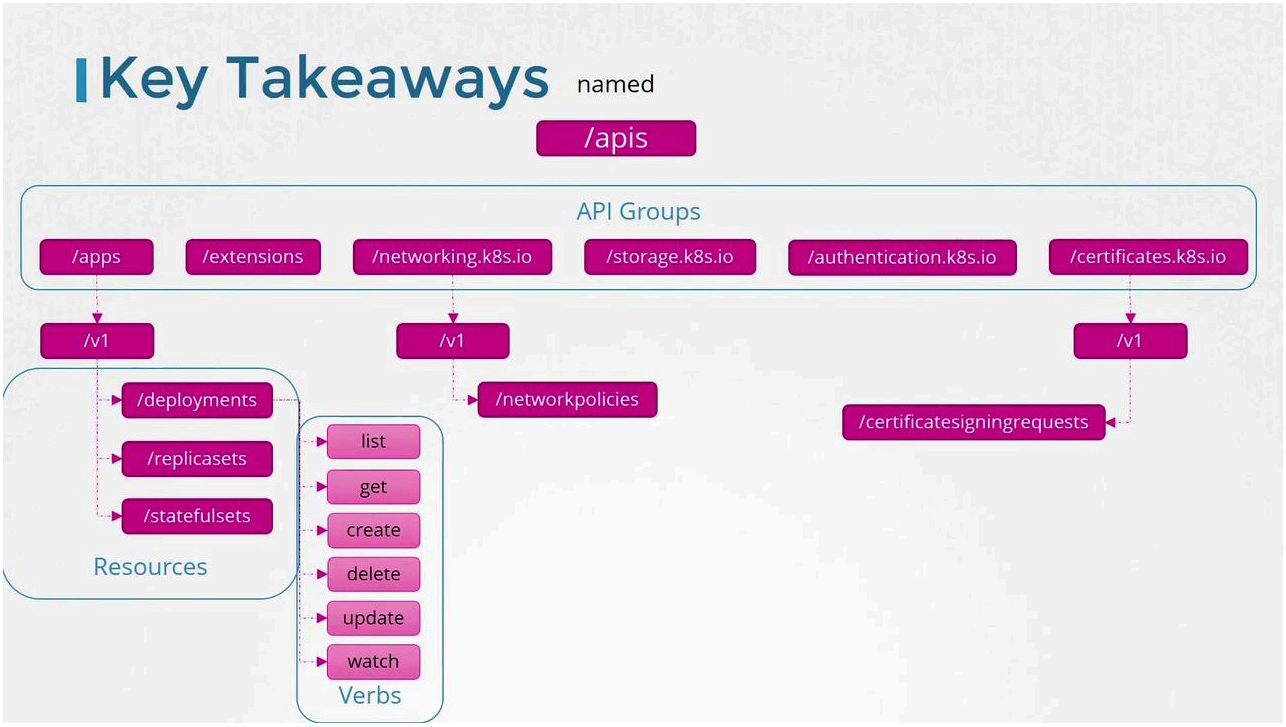
Remember that kube-proxy and kubectl proxy are distinct components. The kube-proxy facilitates network communication between pods and services across cluster nodes, whereas the kubectl proxy is an HTTP proxy that forwards your requests to the Kubernetes API server using your kubeconfig credentials.

In summary, Kubernetes organizes its resources into different API groups:

- **Core API Group:** Contains fundamental resources such as namespaces, pods, nodes, and services.
- **Named API Groups:** Organize additional and newer functionalities such as deployments, networking, storage, and more.

Each resource within these groups supports a set of verbs (list, get, create, delete, update, watch) defining the operations you can perform.

The diagram below outlines the overall structure and hierarchy of Kubernetes API groups, resources, and verbs:



This concludes our discussion on Kubernetes API groups. In the next lesson, we will continue exploring further aspects of cluster operations.

Watch Video

Watch video content

Previous

KubeConfig

Next

Authorization